

三维力导向图物理仿真 与状态空间可视化系统

基于 GPU 加速的华容道滑块谜题
状态空间探索与可视化平台

技术文档

2026 年 6 月 13 日

摘要

本文档描述了一套用于探索滑块谜题状态空间的三维力导向图可视化系统的设计、数学基础与实现。系统以经典华容道（Klotski）谜题为主要应用对象，包含四个核心模块：(i) 滑块谜题引擎，支持任意矩形棋盘的建模与全状态空间广度优先枚举；(ii) 基于 Taichi 框架的 GPU 加速物理仿真引擎，在数千节点规模上并行计算库仑式斥力与弹簧式引力；(iii) 基于 OpenGL 3.3 Core Profile 的自定义三维渲染器，提供实时交互与自动相机控制；(iv) 异步视频录制管道，将图结构的生长过程与后续环绕观察导出为高质量 MPEG-4 视频。本文详述了力模型的数学推导、基于广度优先搜索（BFS）的图构建算法、基于着色器的深度衰减渲染管线，以及十六种华容道谜题配置及其状态空间统计。

目录

第一章 引言	5
1.1 研究动机	5
1.2 系统概览	5
1.3 相关工作	6
第二章 滑块谜题基础	7
2.1 问题定义	7
2.2 华容道谜题	7
2.3 棋盘表示	7
2.3.1 状态标识	8
2.3.2 邻居枚举	8
2.3.3 有墙变体	9
第三章 状态空间枚举	10
3.1 广度优先搜索	10
3.1.1 复杂度分析	10
3.2 图的表示	11
3.3 示例谜题的状态空间统计	12
第四章 力导向图物理仿真	13
4.1 力模型	13
4.1.1 库仑斥力	13
4.1.2 胡克引力（弹簧模型）	13
4.1.3 中心引力	14
4.1.4 阻尼	14
4.1.5 力平衡分析	14
4.2 基于 Taichi 的 GPU 加速	14
4.2.1 Taichi 内核	14
4.3 BFS 生长仿真	15
4.3.1 状态管理	15
第五章 三维渲染引擎	17
5.1 OpenGL 管线	17

5.1.1	缓冲区管理	17
5.2	着色器系统	17
5.2.1	顶点着色器	17
5.2.2	片段着色器	18
5.3	相机控制	19
5.3.1	球坐标系	19
5.3.2	基于 PCA 主轴分析的相机方位对齐	19
5.3.3	自动距离调整	20
5.3.4	交互控制	20
第六章	视频录制与导出	21
6.1	架构设计	21
6.1.1	帧捕获管线	21
6.2	渲染阶段	21
6.3	信息叠加	22
第七章	集成与使用	23
7.1	依赖项安装	23
7.2	运行系统	23
7.2.1	批量视频导出	23
7.2.2	交互模式	24
7.3	添加新谜题	24
第八章	数学附录	25
8.1	力模型平衡态推导	25
8.2	基于投影的深度衰减原理	25
8.3	BFS 复杂度证明	26
第九章	总结与展望	27
9.1	工作总结	27
9.2	未来方向	27

插图

1.1	系统架构与数据流。谜题状态空间经枚举后被转换为邻接结构，送入力导向物理仿真，经 OpenGL 进行三维渲染，并可选地录制为视频文件。	6
2.1	经典 4×5 华容道初始布局。中央为 2×2 的曹操；五块 1×2 竖块分列两侧；四块 1×1 兵卒占据底部一行。	8
3.1	华容道（谜题 1）状态空间图从根节点出发逐层生长的四个阶段，截图等间距取自自动渲染视频。完整生长与观赏动画见 Bilibili [9] 与 YouTube [10]。	11

表格

3.1 华容道谜题配置及其状态空间规模	12
7.1 软件依赖项	23

第一章 引言

1.1 研究动机

滑块谜题 (sliding-block puzzle)，如华容道 (Klotski)，是一类经典的组合谜题：若干矩形木块须在有限的棋盘区域内移动，以使得指定木块到达目标出口。尽管规则简单，此类谜题的表达能力却令人惊讶——某些配置属于 PSPACE-完全类问题 [1]，意味着在最坏情况下，求解最短路径的复杂度随问题规模呈指数增长。对于典型的含十个以上木块的华容道，不同的棋盘布局（即状态空间）可达数万种。每个状态对应图中的一个节点，每步合法移动对应一条边。理解该图的全局结构——其直径、分支因子、聚类系数以及解路径的几何排布——对于求解算法的设计者与希望欣赏谜题景观的爱好者都具有重要价值。然而，原始的邻接矩阵或二维弹簧布局在节点数超过数百时即变得难以解读。

本项目受 twoswap 的力导向图动画 [11] 启发，将类似的可视化方法推广至滑块谜题的状态空间分析。项目的完整源代码已开源发布于 GitHub [8]，作者制作的演示视频可在 Bilibili [9] 与 YouTube [10] 上观看。

这促使我们构建一个三维力导向可视化系统，以实现以下目标：

1. 自动枚举给定谜题的完整状态空间；
2. 通过精心调校的力导向模型在三维空间中布局生成的图，使聚类与桥接结构在视觉上清晰可辨；
3. 以广度优先遍历方式从起始状态开始逐节点生长图结构，并制作动画；
4. 实时渲染场景，支持三维交互控制，并将动画导出为视频文件。

1.2 系统概览

图 1.1 展示了系统的高层架构。五个 Python 模块以流水线方式协作：

`Klotski.py` 建模滑块谜题棋盘，枚举合法移动，构建全局状态空间图。

`GraphHelper.py` 在图表示形式（邻接矩阵、边列表、邻接表）之间进行转换。

`GraphPhysics.py` 核心物理引擎，通过 Taichi 内核加速库仑斥力计算。维护一个 BFS 驱动的生长模拟，逐步揭示节点。

`GraphRender.py` 基于 OpenGL 3.3 的自定义渲染器，提供 GLSL 着色器、相机控制与自动距离调整。

VideoRecorder.py 异步帧捕获与编码器，叠加进度信息并写入 MPEG-4 视频。

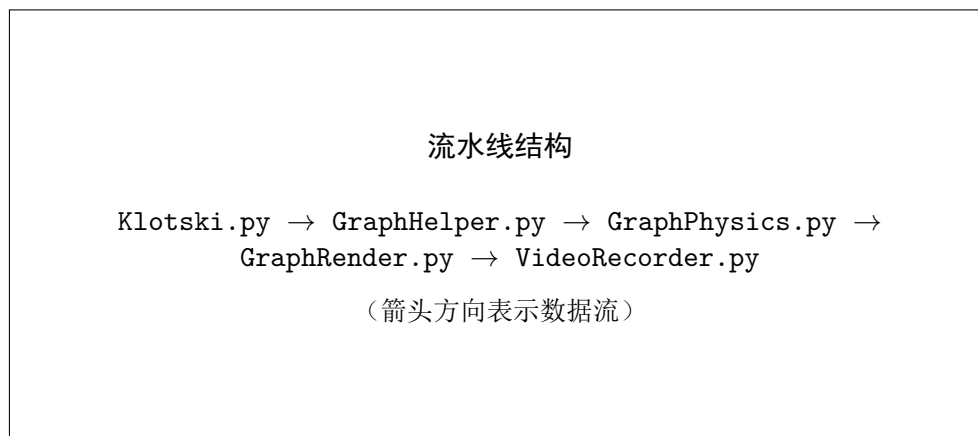


图 1.1: 系统架构与数据流。谜题状态空间经枚举后被转换为邻接结构，送入力导向物理仿真，经 OpenGL 进行三维渲染，并可选地录制为视频文件。

主入口点为 **PlayGround.py**，它实例化选定的谜题配置，构建图，创建物理仿真，并启动交互式查看器或批量视频导出模式。

1.3 相关工作

力导向图布局可追溯至 Eades [2] 与 Fruchterman–Reingold [3] 的开创性工作，至今仍是网络可视化的基石。本文使用的弹簧–电荷模型是 **Graphviz** [4] 和 **networkx** 等流行工具所采用方法的变体。近期研究已将这些技术扩展到三维布局并通过 GPU 加速 [5]，本系统通过 **Taichi** 框架 [6] 采纳了这一方向。滑块谜题的状态空间分析在十五数码问题及其推广中已有研究 [7]。华容道谜题尤其受到多位学者 [1] 的关注，其 PSPACE-完全性已被证明。据我们所知，本系统是首个将完整状态空间枚举、GPU 加速三维力导向布局与视频导出集成于单一工具中的滑块谜题可视化平台。

第二章 滑块谜题基础

2.1 问题定义

滑块谜题由一个 $W \times H$ （宽 $\times$ 高）的矩形棋盘和 B 个矩形木块定义。每个木块 b_i ($i = 0, \dots, B-1$) 的大小为 (w_i, h_i) ，在棋盘上占据位置 (x_i, y_i) ，其中 x_i, y_i 为非负整数，且满足 $0 \leq x_i < W$ 、 $0 \leq y_i < H$ 。一个配置合法当且仅当：

1. 任意两木块不重叠：对任意 $i \neq j$ ，矩形区域 $[x_i, x_i + w_i) \times [y_i, y_i + h_i)$ 与 $[x_j, x_j + w_j) \times [y_j, y_j + h_j)$ 不相交。
2. 每块木块完全位于棋盘内：对所有 i 有 $x_i + w_i \leq W$ 且 $y_i + h_i \leq H$ 。

合法移动指将单个木块沿上、下、左、右四个基本方向之一平移一个单位，且平移后的配置保持合法。**状态空间**是从给定起始配置出发，通过合法移动序列可达的所有配置的集合，连同连接它们的移动本身。

定义 2.1 (状态空间图). 设 $\mathcal{C}$ 为给定谜题的所有合法配置的集合。状态空间图是无向图 $G = (V, E)$ ，其中 $V = \mathcal{C}$ ，且 $(u, v) \in E$ 当且仅当存在一个合法移动将配置 u 变换为配置 v 。

2.2 华容道谜题

经典华容道（图 2.1）使用 4×5 棋盘与 10 块木块：

- 1 块 2×2 大方块（曹操）——必须到达底部的出口。
- 5 块 1×2 竖方块（关羽、张飞、赵云、马超、黄忠）。
- 4 块 1×1 小方块（兵卒）。

2.3 棋盘表示

系统内部将每个配置存储为两个整数数组：

- `box_sizes` ——形状 $(B, 2)$ ，存储各木块的 (w_i, h_i) 。
- `box_positions` ——形状 $(B, 2)$ ，存储各木块的 (x_i, y_i) 。

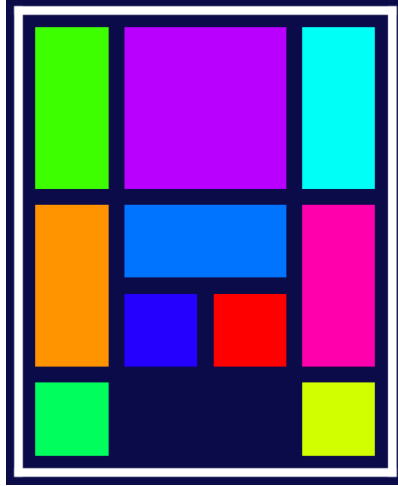


图 2.1: 经典 4×5 华容道初始布局。中央为 2×2 的曹操；五块 1×2 竖块分列两侧；四块 1×1 兵卒占据底部一行。

辅助函数 `get_covering_matrix` 将所有木块栅格化到一张 $H \times W$ 的占用网络上：

$$M_{rc} = \sum_{i=0}^{B-1} \mathbf{1}[x_i \leq c < x_i + w_i \wedge y_i \leq r < y_i + h_i] \quad (2.1)$$

一个配置合法当且仅当对所有 (r, c) 有 $M_{rc} \leq 1$ ，且无木块超出棋盘边界。

2.3.1 状态标识

为高效检测两配置是否相同（BFS 期间的必需操作），我们计算一个标识矩阵，该矩阵对木块的标签不敏感。对每个格子 (r, c) ，其标识值为：

$$I_{rc} = \sum_i \mathbf{1}[\text{木块 } i \text{ 覆盖}(r, c)] \cdot (2^{w_i} \cdot 3^{h_i}) \quad (2.2)$$

此方案为每种尺寸的木块分配一个独特的质数乘积指纹，使得各行构成的元组 $(I_{r0}, \dots, I_{r,W-1})$ 可作为规范哈希。两配置相等当且仅当它们的标识矩阵逐元素相同。

2.3.2 邻居枚举

对每块木块 i ，我们考察四个候选平移方向 $\Delta \in \{\text{上, 下, 左, 右}\}$ ，对应沿各轴的单位位移。一个候选移动合法当且仅当：

1. 平移后的木块不超出棋盘边界。
2. 木块移入和移出的格子不会导致任何格子出现 $M_{rc} > 1$ 。

由于覆盖矩阵始终可用且每次查询为 $\mathcal{O}(1)$ ，每个邻居检查对大小为 (w_i, h_i) 的木块耗时 $\mathcal{O}(w_i + h_i)$ ，因为我们只需检查运动方向上的那些格子。

2.3.3 有墙变体

`WalledSlidingToy` 子类施加了额外的约束：木块只能在其尺寸为 1 的轴向上移动（即 1×2 的木块仅可竖直滑动， 2×1 的木块仅可水平滑动）。此限制模拟了某些物理谜题中由榫接或滑槽限制自由度的情形。

第三章 状态空间枚举

3.1 广度优先搜索

完整状态空间通过标准广度优先搜索进行枚举，如算法 3.1 所示。算法逐层推进，为

```
输入: start_toy (初始 SlidingToy 配置)
      maxiter   (可选的最大 BFS 迭代次数)
输出: adj       (稀疏邻接矩阵, scipy.sparse.csr)
      state2idx (字典: 状态哈希 -> 整数索引)
      idx2state (列表: 索引 -> SlidingToy 实例)
      edges     ((u, v) 无向边元组列表)
state2idx[start] := 0
idx2state[0] := start_toy
current_visitors := [start_toy]
while current_visitors 非空 AND 迭代次数 < maxiter:
    next_visitors := []
    for each 状态 in current_visitors:
        cur_idx := state2idx[hash(状态)]
        for each 邻居 n of 状态:
            if hash(n) not in state2idx:
                state2idx[hash(n)] := |state2idx|
                idx2state.append(n)
                next_visitors.append(n)
            neigh_idx := state2idx[hash(n)]
            if cur_idx < neigh_idx: # 避免重复边
                edges.append((cur_idx, neigh_idx))
    current_visitors := next_visitors
```

Listing 3.1: 构建全局状态空间图

最新发现的状态分配整数索引。边仅记录一次（条件 $\text{cur_idx} < \text{neigh_idx}$ 在无向图中保证了这一点）。最终的邻接矩阵以稀疏 COO 格式构建：

$$A_{ij} = \begin{cases} 1 & \text{若 } (i, j) \in E, \\ 0 & \text{否则.} \end{cases} \quad (3.1)$$

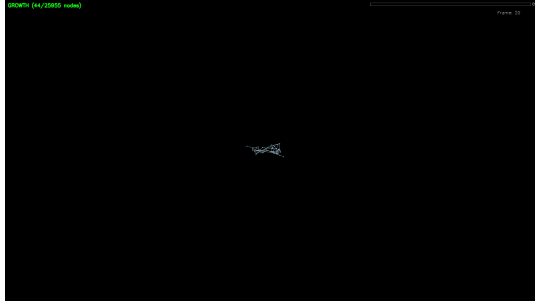
3.1.1 复杂度分析

设 $N = |V|$ 为不同状态数， $E = |E|$ 为边数。每个状态至多生成 $4B$ 个候选邻居（因为 B 块木块每块至多可向四个方向移动）。哈希查找 $\text{hash}(n)$ in state2idx 的摊还成本为

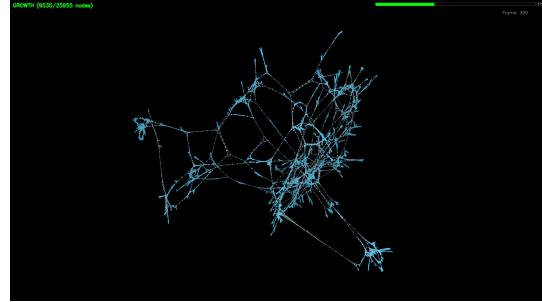
$\mathcal{O}(1)$ 。因此总时间复杂度为：

$$\mathcal{O}(N \cdot B \cdot (w_{\max} + h_{\max}) + E) \quad (3.2)$$

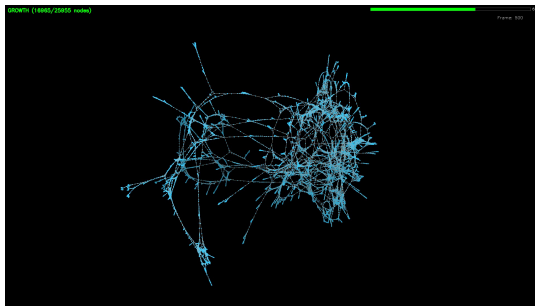
其中 $w_{\max}$ 与 $h_{\max}$ 为影响重叠检查开销的最大木块尺寸。实践中，对于 $N \approx 20\,000$ 、 $B = 10$ 的经典华容道，枚举在主流台式 CPU 上可在两秒内完成。



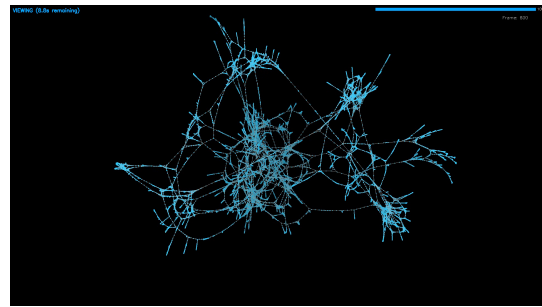
(a) 生长初期，BFS 刚展开数十个节点，图结构稀疏。



(b) 生长约 1/3 处，节点快速增加，核心聚类开始形成。



(c) 生长约 2/3 处，大部分节点已激活，图结构逐渐收敛。



(d) 生长完成，所有 $\sim 20\,000$ 个节点已布局，进入环绕观赏阶段。

图 3.1: 华容道（谜题 1）状态空间图从根节点出发逐层生长的四个阶段，截图等间距取自自动渲染视频。完整生长与观赏动画见 Bilibili [9] 与 YouTube [10]。

3.2 图的表示

枚举得到的图以三种可互换的形式表示：

- **边列表**—— $\mathcal{O}(E)$ 存储，方便构建与序列化。
- **邻接矩阵（稀疏 CSR）**—— $\mathcal{O}(N + E)$ 存储，被 `networkx` 所需，且对矩阵运算高效。
- **邻接表（字典的列表）**—— $\mathcal{O}(N + E)$ 存储，物理仿真使用此形式进行邻居迭代。

三者之间的转换由 `GraphHelper.py` 中的 `edges_to_adjacency_list()` 与 `matrix_to_adjacency_list()` 完成。

3.3 示例谜题的状态空间统计

表 3.1 列出了本系统包含的十六种谜题配置。每种配置均经过穷举枚举（在可行范围内）以获得状态计数。

表 3.1: 华容道谜题配置及其状态空间规模

编号	棋盘	木块数	变体	状态数 (N)
1	4×5	10	经典	$\sim 20\,000$
2	6×6	13	有墙	
3	4×4	3	经典	~ 300
4	4×5	7	经典	
5	4×5	6	经典	
6	4×6	11	经典	
7	4×6	7	经典	
8	4×5	15	经典	
9	4×5	14	经典	
10	6×6	10	有墙	
11	6×6	6	有墙	
12	4×5	9	经典	
13	5×5	7	经典	
14	6×6	13	有墙	
15	6×6	8	有墙	
16	6×6	13	有墙	

第四章 力导向图物理仿真

4.1 力模型

布局通过力导向模型计算，该模型在连续时间 t 上演化节点位置 $\mathbf{p}_i(t) \in \mathbb{R}^3$ 。每个节点受到三类力的作用。

4.1.1 库仑斥力

任意两个不同节点 (i, j) 之间施加类库仑斥力：

$$\mathbf{f}_{i \leftarrow j}^{\text{rep}} = \frac{k_{\text{coul}}^2}{\|\mathbf{p}_i - \mathbf{p}_j\|^2 + \varepsilon} \cdot (\mathbf{p}_i - \mathbf{p}_j) \quad (4.1)$$

其中 k_{coul} 为用户可调常数（默认 25）， $\varepsilon = 10^{-4}$ 为软化参数，防止节点重合时出现奇点。注意到该表达式中 $(\mathbf{p}_i - \mathbf{p}_j)$ 的模长为 $r = \|\mathbf{p}_i - \mathbf{p}_j\|$ ，因此斥力大小为：

$$\|\mathbf{f}_{i \leftarrow j}^{\text{rep}}\| = \frac{k_{\text{coul}}^2}{r^2 + \varepsilon} \cdot r \approx \frac{k_{\text{coul}}^2}{r} \quad (4.2)$$

这是一个 $1/r$ 型的力律，与二维空间中点电荷的电场强度类似。节点 i 上的总斥力为：

$$\mathbf{f}_i^{\text{rep}} = \sum_{j \neq i} \mathbf{f}_{i \leftarrow j}^{\text{rep}} \quad (4.3)$$

4.1.2 胡克引力（弹簧模型）

边对连接的节点施加弹簧式引力。对每条边 (u, v) ：

$$\mathbf{f}_{u \leftarrow v}^{\text{att}} = -\frac{\|\mathbf{p}_u - \mathbf{p}_v\|}{k_{\text{coul}}} \cdot (\mathbf{p}_u - \mathbf{p}_v) \quad (4.4)$$

该力的大小随分离距离平方增长：

$$\|\mathbf{f}_{u \leftarrow v}^{\text{att}}\| = \frac{r}{k_{\text{coul}}} \cdot r = \frac{r^2}{k_{\text{coul}}} \quad (4.5)$$

即引力大小正比于 r^2 （二次弹簧），弹簧常数为 $1/k_{\text{coul}}$ 。这一反直觉的设计（斥力 $1/r$ 而引力 r^2 ）确保连接节点间的平衡距离可控且与 k_{coul} 直接相关，详见第 4.1.5 节。

4.1.3 中心引力

一个弱谐波回复力将所有节点拉向原点：

$$\mathbf{f}_i^{\text{grav}} = -k_{\text{grav}} \cdot \mathbf{p}_i \quad (4.6)$$

默认 $k_{\text{grav}} = 0.02$ 。此力防止图结构在斥力作用下无限漂移。

4.1.4 阻尼

速度比例阻尼确保系统收敛至稳定平衡而非持续振荡：

$$\mathbf{v}_i^{(t+1)} = \beta \left(\mathbf{v}_i^{(t)} + \frac{\mathbf{f}_i}{m_i} \Delta t \right), \quad \mathbf{p}_i^{(t+1)} = \mathbf{p}_i^{(t)} + \mathbf{v}_i^{(t+1)} \Delta t \quad (4.7)$$

阻尼系数 $\beta = 0.8$ ，时间步 $\Delta t = 0.15$ ，所有节点单位质量 ($m_i = 1$)。速度限幅 $\|\mathbf{v}_i\|_{\text{max}} = 10$ 可防止布局初期力较大时出现数值不稳定。

4.1.5 力平衡分析

考虑一对连接节点 (i, j) ，在平衡距离 d_0 处斥力与引力大小相等：

$$\frac{k^2}{d_0} = \frac{d_0^2}{k} \implies d_0^3 = k^3 \implies d_0 = k \quad (4.8)$$

因此连接节点稳定在相距约 k_{coul} 的位置，而非连接节点则被推至更远距离，在视觉上形成清晰的聚类分离。

4.2 基于 Taichi 的 GPU 加速

$\mathcal{O}(N^2)$ 的斥力计算是 $N > 1000$ 节点时的主要计算瓶颈。为实现交互帧率，我们将此计算卸载至 GPU，使用 Taichi——一种嵌入 Python 的领域特定语言，可将内核编译至 Vulkan、CUDA、Metal 或 CPU 后端 [6]。

4.2.1 Taichi 内核

代码 4.1 展示了在一次遍中同时计算斥力与引力的核心内核。

```

1 @ti.kernel
2 def compute_repulsion_taichi(
3     pos:      ti.types.ndarray(dtype=ti.types.vector(3, ti.f32), ndim=1),
4     forces:   ti.types.ndarray(dtype=ti.types.vector(3, ti.f32), ndim=1),
5     pairs:    ti.types.ndarray(dtype=ti.i32, ndim=2),
6     k_sq:     ti.f32,
7     inv_k:    ti.f32,
8 ):
9     n = pos.shape[0]
10    num_pairs = pairs.shape[0]
```



```

11     softening_sq = 1e-4
12     for i in range(n):
13         p_i = pos[i]
14         f_i = ti.Vector([0.0, 0.0, 0.0])
15         for j in range(i + 1, n):
16             diff = p_i - pos[j]
17             dist_sq = diff.norm_sqr() + softening_sq
18             f = (k_sq / dist_sq) * diff
19             f_i += f
20             forces[j] -= f
21         forces[i] += f_i
22     for p in range(num_pairs):
23         idx1 = pairs[p, 0]
24         idx2 = pairs[p, 1]
25         diff = pos[idx1] - pos[idx2]
26         dist = diff.norm() + 1e-6
27         f_grad = -(dist * inv_k) * diff
28         forces[idx1] += f_grad
29         forces[idx2] -= f_grad

```

Listing 4.1: 用于并行力计算的 Taichi 内核

内核每物理迭代从主机端调用一次：

```

1 compute_repulsion_taichi(pos, repulsion_forces, pairs, self.k_sq, 1/
    self.k)

```

Taichi 自动将最外层 `for i in range(n)` 循环映射至 GPU 线程，在可用核心数上实现近线性加速。

4.3 BFS 生长仿真

`BFSGraphSimulation3D` 类管理图的增量生长，用于动画制作。从单一根节点出发，它反复从 BFS 队列中弹出一节点并激活其所有未访问的邻居。新节点在其父节点附近生成，施加随机三维扰动，然后在力模型作用下逐步趋于稳定：

```

1 random_dir = np.random.normal(0, 1, 3)
2 random_dir /= np.linalg.norm(random_dir)
3 self.pos_array = np.vstack([self.pos_array, parent_pos + random_dir *
    self.k])

```

这种方式确保观察者看到图从根节点向外组装，模拟经典的 BFS 可视化过程。

4.3.1 状态管理

仿真维护以下数据结构：

- `pos_array / vel_array` —— $(N, 3)$ numpy 数组，作为节点位置与速度的主存储。

- `active_edges` —— 两端点均已被激活的边的 (u, v) 元组集合。
- `pairs_array` —— 缓存的边索引对数组，供 Taichi 内核使用。
- `bfs_queue` —— 一个 `collections.deque`，存储尚待扩展的节点索引。
- `visited` —— 已入队的节点索引集合（防止重复添加）。

每步生长之后，物理引擎运行数次迭代（通常 3–6 次），使新插入的节点在下次添加之前部分松弛。这种生长与松弛的交织创造了视觉上平滑的涌现效果。

第五章 三维渲染引擎

5.1 OpenGL 管线

渲染器基于 OpenGL 3.3 Core Profile 构建，通过 PyOpenGL 和 GLFW 进行窗口管理。管线遵循标准现代 OpenGL 流程：

1. 顶点数据（位置）通过 VBO 上传至 GPU，与 VAO 关联。
2. 自定义 GLSL 着色器程序对顶点进行变换与着色。
3. 点以 GL_POINTS 图元绘制，边以 GL_LINES 绘制。

系统维护两套独立的 VAO/VBO 对：一套用于点位置，一套用于边顶点（每条边贡献两个顶点作为线段）。这种分离允许在图生长时独立更新各缓冲区。

5.1.1 缓冲区管理

每帧通过以下方式更新缓冲区：

```
1 glBindBuffer(GL_ARRAY_BUFFER, self.point_vbo)
2 glBufferData(GL_ARRAY_BUFFER, points.nbytes, points, GL_DYNAMIC_DRAW)
3 glBindBuffer(GL_ARRAY_BUFFER, self.edge_vbo)
4 glBufferData(GL_ARRAY_BUFFER, edges.nbytes, edges, GL_DYNAMIC_DRAW)
```

GL_DYNAMIC_DRAW 用法提示向驱动程序发出信号：顶点数据会频繁变化，从而优化上传带宽。

5.2 着色器系统

5.2.1 顶点着色器

顶点着色器（代码 5.1）将位置变换至裁剪空间，并计算每个顶点的投影值 v_{proj} ：

$$v_{\text{proj}} = \mathbf{p} \cdot (-\mathbf{d}_{\text{look}}) \quad (5.1)$$

其中 $\mathbf{d}_{\text{look}}$ 为归一化的相机观察方向。该值表示顶点到垂直于观察方向的平面的有符号距离。

```

1 #version 330 core
2 layout (location = 0) in vec3 aPos;
3 uniform mat4 view;
4 uniform mat4 projection;
5 uniform vec3 camLookDir;
6 out float v_proj;
7 void main() {
8     vec4 viewPos = view * vec4(aPos, 1.0);
9     v_proj = dot(aPos, -camLookDir);
10    gl_Position = projection * viewPos;
11 }

```

Listing 5.1: GLSL 顶点着色器

5.2.2 片段着色器

片段着色器（代码 5.2）应用基于深度的强度衰减。远离相机的节点和边显得更暗，增强了三维深度感知。

```

1 #version 330 core
2 out vec4 FragColor;
3 in float v_proj;
4 uniform vec3 color;
5 uniform float near;
6 uniform float far;
7 void main() {
8     float range = near - far;
9     float intensity = (range > 0.001)
10        ? clamp((v_proj - far) / range, 0.0, 1.0)
11        : 1.0;
12    FragColor = vec4(color * intensity, 1.0);
13 }

```

Listing 5.2: 带深度衰减的 GLSL 片段着色器

near 和 far 统一变量根据当前点云投影范围动态设定：

```

1 projections = np.dot(self.points, cam_look_dir_np)
2 near_val = np.max(projections) / 2
3 far_val = np.min(projections) * 3

```

这种动态范围映射确保深度提示适应图的当前尺度。

5.3 相机控制

5.3.1 球坐标系

相机被置于以目标点 $\mathbf{t} \in \mathbb{R}^3$ 为中心、半径为 d 的球面上，由偏航角 θ 和俯仰角 ϕ 参数化：

$$\mathbf{p}_{\text{cam}} = \mathbf{t} + d \begin{pmatrix} \cos \theta \cos \phi \\ \sin \phi \\ \sin \theta \cos \phi \end{pmatrix} \quad (5.2)$$

视图矩阵通过 `glm::lookAt` 生成。默认的世界向上向量为 $(0, 1, 0)$ ，但在 PCA 对齐模式下（第 5.3.2 节），向上向量与 look 方向均由节点分布的协方差主轴确定，使得主成分沿屏幕横向展开。

5.3.2 基于 PCA 主轴分析的相机方位对齐

当力导向布局收敛后，图结构往往沿若干主轴方向伸展。若相机方位与这些主轴错位，则大部分有价值的结构信息可能被压缩在屏幕的狭窄区域内，造成视觉混淆。为获得信息量最大的观察视角，系统对节点位置进行主成分分析（PCA），并将相机对准图的次长主轴。

设节点位置矩阵 $\mathbf{P} \in \mathbb{R}^{N \times 3}$ ，其中第 i 行为 $\mathbf{p}_i$ 。去中心化后计算协方差矩阵：

$$\mathbf{\Sigma} = \frac{1}{N} \sum_{i=1}^N (\mathbf{p}_i - \bar{\mathbf{p}})(\mathbf{p}_i - \bar{\mathbf{p}})^T \in \mathbb{R}^{3 \times 3} \quad (5.3)$$

其中 $\bar{\mathbf{p}} = \frac{1}{N} \sum_i \mathbf{p}_i$ 为质心。对 $\mathbf{\Sigma}$ 进行特征分解：

$$\mathbf{\Sigma} \mathbf{v}_k = \lambda_k \mathbf{v}_k, \quad \lambda_0 \geq \lambda_1 \geq \lambda_2 \geq 0 \quad (5.4)$$

三个特征向量 $\mathbf{v}_0, \mathbf{v}_1, \mathbf{v}_2$ 分别对应主成分（最大方差方向）、次成分和第三成分。系统将相机对准次成分方向：

- **look** 方向 $\mathbf{d}_{\text{look}} \parallel \mathbf{v}_1$ ：摄像机视线沿次成分，使得主成分在屏幕平面上横向展开。
- **up** 方向 $\mathbf{d}_{\text{up}} \parallel \mathbf{v}_2$ ：第三成分作为世界向上向量，确保第三维与屏幕垂直方向对齐。

为防止相机方位突变引起视觉不适，系统对方向向量执行线性插值平滑过渡。设当前 look 方向为 $\mathbf{d}_{\text{look}}^{(t)}$ ，目标方向为 $\mathbf{d}_{\text{look}}^* = \mathbf{v}_1 / \|\mathbf{v}_1\|$ ，更新规则为：

$$\mathbf{d}_{\text{look}}^{(t+1)} = \text{norm} \left(\mathbf{d}_{\text{look}}^{(t)} + \alpha (\mathbf{d}_{\text{look}}^* - \mathbf{d}_{\text{look}}^{(t)}) \right), \quad \alpha = 0.04 \quad (5.5)$$

其中 $\text{norm}(\cdot)$ 为单位归一化。 α 取小值确保数百帧内逐步收敛。在生长阶段，PCA 仅在节点数量变化时重新计算，避免每帧 $\mathcal{O}(N)$ 的协方差矩阵分解开销。

5.3.3 自动距离调整

为使图在生长过程中始终可见，每帧使用综合考虑包围盒与节点数的混合指标更新相机距离：

$$d_{\text{ideal}} = \underbrace{\max \left(\max_i \|\mathbf{p}_i - \bar{\mathbf{p}}\|, 0.9 \cdot \text{bbox} \right)}_{\text{场景尺度}} + \underbrace{0.5 \cdot \ln(N + 1)}_{\text{节点因子}} \quad (5.6)$$

实际距离通过指数移动平均平滑：

$$d^{(t+1)} = 0.92 \cdot d^{(t)} + 0.08 \cdot d_{\text{ideal}} \quad (5.7)$$

这既防止了分散注意力的突变，又使图保持良好的取景。

5.3.4 交互控制

当渲染器以 `visible=True` 运行时，用户可通过鼠标控制相机：

- **拖拽**（左键）——调整偏航与俯仰。
- **滚轮**——调整距离（拉近/拉远）。
- **ESC**——退出交互循环。

相机回调通过 `GLFW` 注册，实时更新内部偏航/俯仰/距离参数。

第六章 视频录制与导出

6.1 架构设计

`AutoVideoRecorder` 类处理异步帧捕获与编码，将渲染循环与磁盘 I/O 解耦，防止丢帧。设计采用生产者-消费者模式：

1. 渲染线程（生产者）每帧调用 `capture_frame()`，通过 `glReadPixels` 读取 OpenGL 前端缓冲区，叠加信息层，并将帧推入最大长度 200 的双端队列。
2. 后台线程（消费者）持续从队列中取出帧并写入 `OpenCV VideoWriter` 对象。

6.1.1 帧捕获管线

每帧捕获后经历以下变换：

1. 读取： `glReadPixels` 从 OpenGL 前端缓冲区提取 RGB 像素。
2. 重塑： 原始字节缓冲区被重塑为 $H \times W \times 3$ 的 `uint8` 数组。
3. 翻转： OpenGL 原点在左下角，OpenCV 期望左上角。垂直翻转（`cv2.flip(frame, 0)`）予以纠正。
4. 转换： `cv2.cvtColor(..., COLOR_RGB2BGR)` 将 OpenGL 的 RGB 转换为 OpenCV 的 BGR 通道顺序。
5. 叠加： 顶部半透明黑条显示当前阶段、节点数和进度条。

6.2 渲染阶段

录制器管理一个三阶段状态机：

GROWTH（生长） BFS 仿真每帧添加节点。覆盖层显示 “GROWTH (m/N 节点)” 及绿色进度条。

VIEWING（观赏） 所有节点均已激活；相机在指定时长内环绕完整的图。覆盖层转为橙色并显示倒计时。

FINISHED（完成） 停止录制并释放资源。

当激活节点数达到总数时自动触发阶段转换。

6.3 信息叠加

代码 6.1 展示了使用 OpenCV 绘图原语的覆盖层绘制代码。

```
1 def _add_overlay(self, frame):
2     h, w = frame.shape[:2]
3     frame[:60, :] = (frame[:60, :].astype(np.float32) * 0.7).astype(np.
4         uint8)
5     if phase == GROWTH:
6         text = f"GROWTH ({current}/{total} nodes)"
7         color = (0, 255, 0)
8     elif phase == VIEWING:
9         text = f"VIEWING ({t:.1f}s remaining)"
10        color = (255, 165, 0)
11    cv2.putText(frame, text, (10, 25),
12        cv2.FONT_HERSHEY_SIMPLEX, 0.6, color, 2)
13    cv2.rectangle(frame, (bar_x, 10), (bar_x + bar_width, 20), ...)
14    cv2.rectangle(frame, (bar_x, 10),
15        (bar_x + int(bar_width * progress), 20), color, -1)
```

Listing 6.1: 含进度条的信息叠加层渲染

第七章 集成与使用

7.1 依赖项安装

系统依赖以下软件包，列于 `requirements.txt`：安装命令：

表 7.1: 软件依赖项

包名	版本	用途
taichi	$\geq 1.7.0$	GPU 加速物理内核
numpy	—	数值数组
scipy	—	稀疏邻接矩阵
networkx	—	图算法与布局
glfw	—	窗口管理与输入
PyOpenGL	—	OpenGL 绑定
PyGLM	—	相机矩阵线性代数
opencv-python	—	视频编码与叠加层
matplotlib	—	棋盘示意图（调试/工具）

```
1 pip install -r requirements.txt
```

7.2 运行系统

7.2.1 批量视频导出

默认模式会生成一段图从根节点逐步生长的视频。

```
1 start_toy = SlidingToy(width=4, height=5, ...)
2 _, _, _, edges = build_global_graph(start_toy)
3 adj_list = edges_to_adjacency_list(edges)
4 sim = BFSGraphSimulation3D(adj_list, root_node=0, k=0.05, ...)
5 renderer = GraphRenderer((1920, 1080), visible=False)
6 renderer.render_video(sim, "output.mp4",
7                         viewing_duration=10.0,
8                         nodes_per_frame=100,
9                         fps=60)
```

7.2.2 交互模式

如需实时探索，设置 `visible=True` 并调用 `run_interactive`:

```
1 renderer = GraphRenderer((1280, 720), visible=True)
2 renderer.run_interactive(sim)
```

交互窗口支持基于鼠标的相机控制与 ESC 退出。

7.3 添加新谜题

如需添加自定义谜题，定义棋盘尺寸与木块布局:

```
1 my_puzzle = SlidingToy(
2     width=4, height=5,
3     box_sizes=np.array([
4         [2, 2], [1, 2], [1, 2], [1, 2], [1, 2],
5         [2, 1], [1, 1], [1, 1], [1, 1], [1, 1],
6     ], dtype=int),
7     box_positions=np.array([
8         [1, 0], [0, 0], [0, 2], [3, 0], [3, 2],
9         [1, 2], [0, 4], [3, 4], [1, 3], [2, 3],
10    ], dtype=int)
11 )
```

对于受约束移动的谜题（有墙木块），使用 `WalledSlidingToy`。

第八章 数学附录

8.1 力模型平衡态推导

我们推导与单一邻居 j 相连的节点 i 所受合力的平衡条件。令 $r = \|\mathbf{p}_i - \mathbf{p}_j\|$ 。 j 对 i 的斥力大小为：

$$F_{\text{rep}} = \frac{k^2}{r} \quad (8.1)$$

弹簧引力大小为：

$$F_{\text{att}} = \frac{r^2}{k} \quad (8.2)$$

平衡时 $F_{\text{rep}} = F_{\text{att}}$ ，可得：

$$\frac{k^2}{r} = \frac{r^2}{k} \implies r^3 = k^3 \implies r = k \quad (8.3)$$

对于度为 d_i 的节点 i ，来自所有邻居的力相互叠加。在对称性假设下忽略非邻居的斥力（此为简化），平衡距离近似为：

$$r_{\text{eq}} \approx k \cdot d_i^{-1/3} \quad (8.4)$$

这表明高度数节点更靠近其邻居，而低度数节点被推至更远——这一理想特性将枢纽节点拉入聚类中心。

8.2 基于投影的深度衰减原理

片段着色器使用投影值 $v_{\text{proj}} = \mathbf{p} \cdot (-\mathbf{d}_{\text{look}})$ 计算逐片段强度 I ：

$$I = \text{clamp}\left(\frac{v_{\text{proj}} - f}{n - f}, 0, 1\right) \quad (8.5)$$

其中 $n = \frac{1}{2} \max(v_{\text{proj}})$ 与 $f = 3 \cdot \min(v_{\text{proj}})$ 为动态计算的远近裁剪参考。由于 v_{proj} 是沿观察方向的投影距离，该映射呈线性：靠近相机的点（ v_{proj} 高）获得 $I \approx 1$ ，远离相机的点（ v_{proj} 低）则衰减至 $I = 0$ 。此方法比真正的雾效或基于 Z 缓冲的方案计算成本更低，且能自动适应当前图的尺度，无需用户指定远近平面。

8.3 BFS 复杂度证明

定理 8.1 (状态空间 BFS 复杂度). 设 $G = (V, E)$ 为 $W \times H$ 滑块谜题的状态空间图, 共 B 块木块, 最大尺寸 $d_{\max} = \max(w_{\max}, h_{\max})$ 。BFS 枚举 (算法 3.1) 的时间复杂度为 $\mathcal{O}(N \cdot B \cdot d_{\max} + E)$, 其中 $N = |V|$ 。

证明. 对于 N 个状态中的每个状态, 我们生成至多 $4B$ 个候选邻居。每个候选邻居需要一次重叠检查, 该检查最多考察覆盖矩阵的 $\mathcal{O}(w_i + h_i) = \mathcal{O}(d_{\max})$ 个格子。因此邻居生成成本为 $\mathcal{O}(N \cdot B \cdot d_{\max})$ 。每条发现的边以 $\mathcal{O}(1)$ 时间加入列表, 贡献 $\mathcal{O}(E)$ 。状态去重的哈希查找摊还成本为 $\mathcal{O}(1)$ 。总和即得所述复杂度。□

推论 8.1. 对于 $B = 10$ 、 $d_{\max} = 2$ 、 $N \approx 20\,000$ 的经典 4×5 华容道, 预期运行时间约为 $\mathcal{O}(2 \times 10^5)$ 次操作, 在 3 GHz CPU 上可在 1 秒内完成。

第九章 总结与展望

9.1 工作总结

我们提出了一个完整的系统，用于在三维空间中枚举、仿真、渲染和记录滑块谜题的状态空间图。主要贡献包括：

1. 一个通用的滑块谜题框架，支持任意棋盘几何、木块排布与移动约束。
2. 一个基于 GPU 加速的力导向物理引擎，在 $\sim 10\,000$ 节点规模下实现交互性能。
3. 一个自定义 OpenGL 渲染器，具备动态深度衰减与自动相机控制。
4. 一个异步视频录制管道，可产出带有信息叠加层的高质量输出。

9.2 未来方向

以下扩展方向值得探索：

多 GPU 支持 可扩展 Taichi 内核，将 $\mathcal{O}(N^2)$ 斥力计算分布到多个 GPU 上，以支持更大规模的图。

交互式编辑 允许用户点击节点，在侧栏中查看对应的棋盘配置。

路径高亮 给定起始与目标配置，计算并在 3D 布局中高亮最短路径。

立体渲染 为虚拟现实头显渲染瞳距偏移的两个视图，实现沉浸式探索。

Web 部署 将 Taichi 内核编译至 WebAssembly，通过 WebGL 在浏览器中部署可视化器。

参考文献

- [1] R. A. Hearn and E. D. Demaine. “PSPACE-completeness of sliding-block puzzles and other problems through the nondeterministic constraint logic model of computation.” *Master’s thesis, MIT*, 2006.
- [2] P. Eades. “A heuristic for graph drawing.” *Congressus Numerantium*, 42:149–160, 1984.
- [3] T. M. J. Fruchterman and E. M. Reingold. “Graph drawing by force-directed placement.” *Software: Practice and Experience*, 21(11):1129–1164, 1991.
- [4] J. Ellson, E. Gansner, L. Koutsofios, S. C. North, and G. Woodhull. “Graphviz — open source graph drawing tools.” *Graph Drawing*, pages 483–484, 2001.
- [5] Y. Liu, J. Tang, and W. Wang. “GPU-accelerated force-directed graph drawing.” *IEEE Pacific Visualization Symposium*, pages 231–234, 2014.
- [6] Y. Hu, T.-M. Li, L. Anderson, J. Ragan-Kelley, and F. Durand. “Taichi: a language for high-performance computation on spatially sparse data structures.” *ACM Transactions on Graphics (TOG)*, 38(6):1–16, 2019.
- [7] D. Ratner and M. Warmuth. “Finding a shortest solution for the $N \times N$ extension of the 15-puzzle is intractable.” *Proceedings of AAAI*, pages 168–172, 1986.
- [8] Physics-Based 3D Graph Visualizer. <https://github.com/AStarySky/physics-based-3D-graph-visualizer>
- [9] 16 个不同华容道游戏的解空间. <https://www.bilibili.com/video/BV1N1QTB4EZt>
- [10] 16 Different Klotski Graph Visualized. <https://www.youtube.com/watch?v=4orthiDjWME>
- [11] twoswap. “I solved Klotski” (力导向图动画灵感来源). <https://www.youtube.com/watch?v=YGLNyHd2w10>